Migration Map

Generated at: 7/30/2025, 11:40:50 AM

Files	Migration Plan	Suggested Tech	Database Suggestion	Note
config.php	1. Convert the PHP code in config.php to JavaScript/React syntax. This file may contain configuration settings that can be refactored into a JSON file in the target system. 2. Create separate modules/ components in React to handle configuration settings if needed.	For handling configuration settings in React, consider using a library like dotenv for environment variables or creating a JSON file for configuration settings.	For the database schema, consider using an ORM (Object-Relational Mapping) library like Sequelize or TypeORM in the target system to interact with the database. These libraries can help in defining models, relationships, and executing queries in a more structured way.	When migrating from PHP to React, keep in mind that React is a front-end library for building user interfaces, while PHP is a server-side language. Make sure to separate concerns properly and utilize React for the front-end logic and PHP (or another back-end language) for server-side operations.
PaymentService.php UserController.php User.php	1. Refactor the PHP files into separate components in ReactJS, following a component-based architecture. For example, create a User component, PaymentService component, and UserController component. Ensure that the dependencies are properly managed between these	For managing state and data flow in ReactJS, consider using Redux or React Context API. For making API calls, use Axios or fetch API.	Consider migrating the database schema to a modern ORM like Sequelize for Node.js. This will help in managing database interactions more efficiently and provide a more structured approach to handling data.	When migrating from PHP to ReactJS, keep in mind that React is a front-end library and does not handle server-side logic like PHP. You will need to set up a separate backend server (e.g., Node.js with Express) to handle server-side operations and API calls. Ensure proper data validation and error handling in the new system.

Files	Migration Plan	Suggested Tech	Database Suggestion	Note
	components.			
schema.sql	1. Convert the database schema from SQL to a NoSQL database like MongoDB for better scalability and flexibility. 2. Refactor the PHP code to React components that interact with the MongoDB database using a library like Mongoose. 3. Create separate React components for users and payments, each handling their respective CRUD operations.	Use MongoDB as the database solution and Mongoose as the ORM for interacting with the database in the React application.	Migrate the SQL schema to a MongoDB collection structure, with separate collections for users and payments. Use Mongoose to define schemas and models for each collection, and handle database operations in the React components.	When converting from PHP to React, keep in mind that React is a front-end library and does not handle server-side logic like PHP. You will need to set up a separate back-end server (e.g., Node.js) to handle API requests and interact with the database. Make sure to refactor the PHP code into reusable React components and utilize state and props to manage data flow within the application.